

House Price Prediction from Specs and Exterior Photos

CMPS 4450 Data Mining Final Project

California State University, Bakersfield · Dr. Chengwei Lei

Noah Gallego · Gerardo Gomez · Justin Lo · Juancarlos Sandoval
Group 4

May 2026

Abstract

We investigate whether the exterior photograph of a house, summarized as hand-computed numerical features, can improve price prediction beyond basic property specifications (bedrooms, bathrooms, square footage, and city). Using 15,474 Southern California house listings, we fit multiple linear regression and K-nearest neighbors on specs-only features, then add 27 simple image statistics and separately 96 spatially-pooled filter responses. We also test preprocessing variants (center cropping, per-image normalization, HSV channels). Validation and a 500-trial shuffle cross-validation both show that adding image features does not improve prediction. The best model on the held-out test set is KNN with Manhattan distance at $k=15$ on specs alone, achieving a mean absolute error of \$150,186 with 51.0% of predictions within \$100,000. We discuss why hand-crafted image features from small exterior photos fail to capture price-relevant visual information.

1 Introduction

Predicting a house's sale price from observable attributes is a well-studied regression problem. Traditional approaches rely on tabular features: bedrooms, bathrooms, square footage, and location. These carry clear, measurable signals about value. However, a house is more than its specifications. The exterior photograph encodes information about condition, landscaping, architectural style, lot size, and neighborhood character that no single number can capture.

This project asks a focused question: given that we already have a working model built on specs, can we extract additional predictive value from the exterior photo? If the photo carries price-relevant signal that the specs miss, adding image-derived features should reduce prediction error. If not, the specs alone suffice.

We restrict our approach to hand-computed numerical features extracted with vanilla Python (numpy, PIL, scipy). We do not use deep learning, pretrained CNNs, or learned embeddings. This keeps every feature interpretable and the project within course scope, but it also sets an upper bound on how much visual information we can extract.

2 Dataset

The dataset is “House Prices and Images – SoCal” (ted8080, 2023), containing 15,474 Southern California house listings. Each listing has one exterior photograph (typically 128×128 pixels) and eight tabular fields: `image_id`, `street`, `citi` (city name), `n_citi` (label-encoded city), `bed`, `bath`, `sqft`, and `price`. There are no missing values.

Sale prices range from \$195,000 to \$2,000,000 (mean \$703,556, median \$639,000). The Kaggle source clipped the long tail at \$2M, creating a hard ceiling. We regress on raw price in dollars without a log transform, so all error metrics remain directly interpretable.

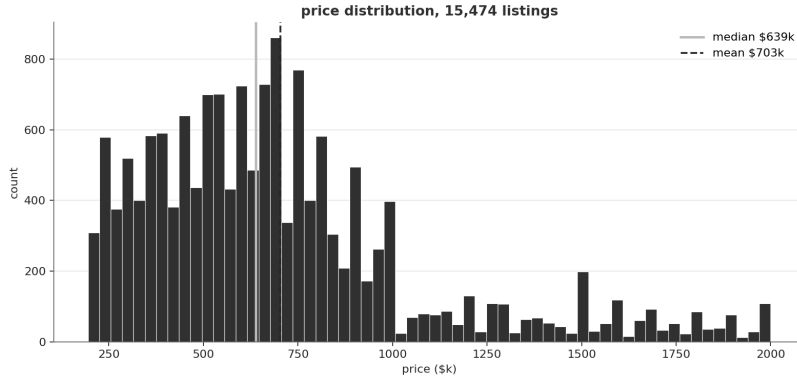


Figure 1: Distribution of sale prices across 15,474 listings.

There are 415 unique cities. The top 10 account for roughly 25% of listings; most cities appear fewer than 50 times. This long-tail distribution makes one-hot encoding impractical, so we encode city identity through geographic coordinates (Section 3).

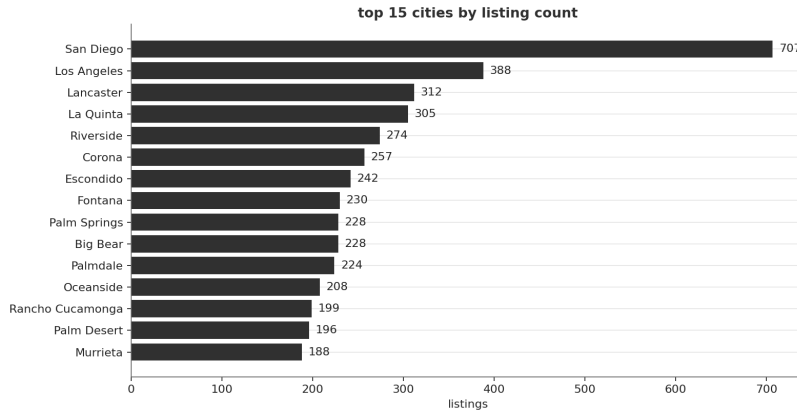


Figure 2: Listing counts for the top 20 cities.

Each row pairs a small exterior photo with a few numeric fields. Figure 3 shows sample listings. The photos vary considerably in angle, lighting, and framing. Some show the full facade; others focus on landscaping, driveways, or even non-house objects (see Figure 11 in the Discussion).

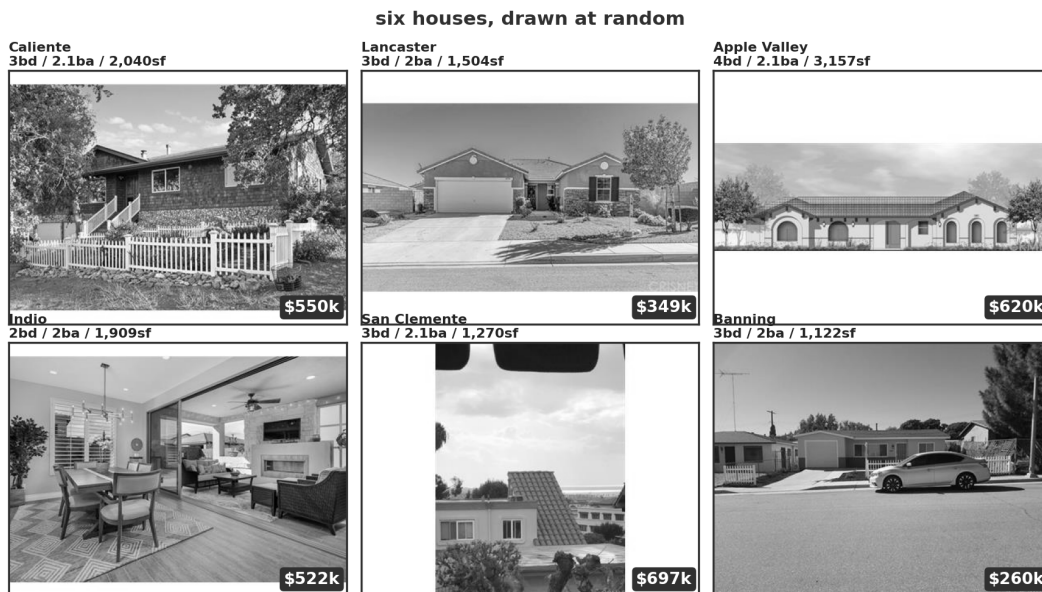


Figure 3: Sample house listings with their specifications and photos.

3 Methodology

3.1 Train / Validation / Test Split

We shuffle the 15,474 listings (random seed 42) and split 65/15/20 into training ($\approx 10,058$ rows), validation ($\approx 2,321$ rows), and test ($\approx 3,095$ rows). Training fits parameters; validation informs all modeling decisions (feature selection, model class, hyperparameters); test is evaluated once after all decisions are locked. To confirm stability, we also run 500-trial shuffle cross-validation: each trial re-shuffles, re-splits, re-fits, and re-evaluates.

3.2 Feature Selection and Engineering

From the eight original columns, we keep `bed`, `bath`, `sqft`, and `price` (target). We drop three columns and replace one with derived features. Each decision is justified below.

Drop `image_id`. This is a sequential row number with no relationship to price. Including it would let the model memorize index-to-price mappings that do not generalize.

Drop `street`. The street address is near-unique per listing (e.g., “1317 Van Buren Avenue”). With 15,474 listings and roughly as many unique street strings, this column acts as an identifier rather than a feature. A model could overfit to individual street names, but there are too few examples per street to learn a reliable price signal.

Drop `n_citi`. This column is an integer label encoding: the dataset assigns each city a unique number from 0 to 414. The problem is that these numbers are arbitrary. City 317 is not “317 times more expensive” than city 0, and the numerical ordering has no meaning. Feeding an arbitrary integer into a linear model tells the model to use city as a single slope, which is incorrect. Feeding it into KNN tells the model that city 0 and city 1 are “close” while city 0 and city 414 are “far,”

which is also wrong. We drop this column entirely.

Replace `citi` with coordinates. The city name does carry price signal: houses in Beverly Hills cost more than houses in Lancaster, on average. But we cannot feed 415 city names directly into a regression model. One-hot encoding would create 415 binary columns, which is impractical given our training set of roughly 10,000 rows (the ratio of features to examples would be too high, and most columns would be near-zero for most rows). Instead, we convert each city name into two continuous numbers that capture *where* that city is.

We geocode each of the 415 unique city names by looking up its real-world latitude and longitude through a geocoding service. We make 415 API calls once and cache the results to disk. Each house then receives two continuous features: `city_lat` and `city_lon`. The advantage is that cities that are physically close to each other receive similar coordinate values, so KNN’s distance metric naturally groups nearby cities together. A house in San Diego (latitude 32.7) and a house in Chula Vista (latitude 32.6) are “close” in feature space because they are close on the map, even though they have different city names. This would not happen with one-hot encoding or label encoding.

To capture broader regional structure, we also cluster the 415 coordinate pairs into 10 regions using K-means on the lat/lon values, selecting $k=10$ by the elbow method (Figure 4). Each house receives a third city-derived feature, `region_id`, indicating which of the 10 geographic clusters its city belongs to. Cities not present in the training set receive the global training-set mean for lat/lon and `region_id`= -1.

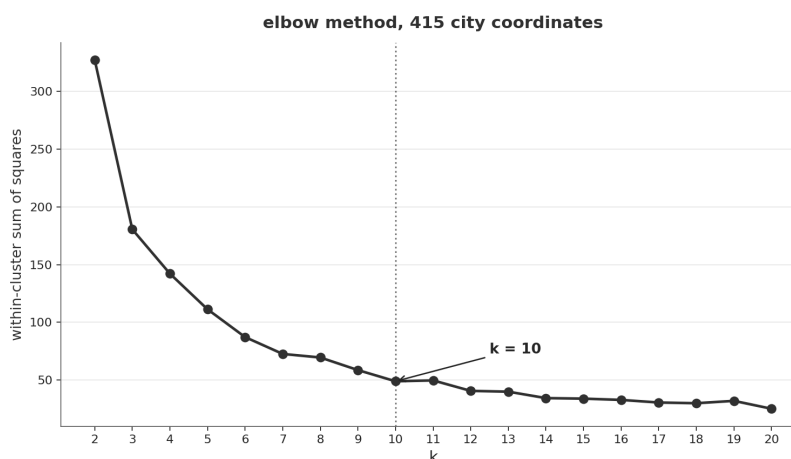


Figure 4: Elbow plot for K-means clustering on 415 city coordinates, supporting $k=10$.

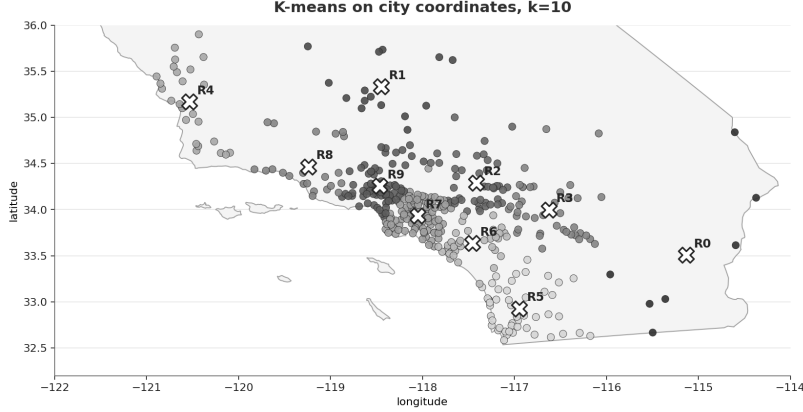


Figure 5: Map of Southern California cities colored by K-means region assignment ($k=10$).

Simple image features (27). We compute 27 hand-crafted numeric features from each exterior photo: mean RGB, grayscale brightness and contrast, Sobel edge density, 4-bin color histograms per channel (12 features), sky brightness and blue ratio, green fraction, HSV saturation mean, pixel variance, center dark ratio, and edge distributions (top vs. bottom).

Spatial filter features (96). The 27 simple features are global statistics that discard spatial structure. To retain layout information, we apply six 3×3 filter kernels (horizontal edge, vertical edge, diagonal edge, center-surround, two texture detectors), convolve each with the grayscale image, then pool each 128×128 response map into a 4×4 grid by averaging each 32×32 patch. Six filters $\times$ 16 pool cells = 96 spatially-aware features. Combined with 4 tabular features, the full set is 100 numbers per house.



Figure 6: Six filter kernels and their response maps on a sample house photo.

3.3 Normalization and Preprocessing Variants

We apply min-max normalization to rescale every feature to $[0, 1]$ using training-set statistics. This is important because raw feature values span orders of magnitude: `sqft` ranges from 500 to 5,400 while `edge_density` is typically 0.01 to 0.2. Without normalization, high-magnitude features would dominate distance calculations in KNN.

After observing that raw image features did not improve the baseline, we tested three preprocessing modifications:

- **Center Crop:** keep only the central 60% of the image (addresses sky/pavement dominance).
- **Per-Image Normalization:** rescale each photo to match the dataset’s mean brightness.
- **HSV Channels:** add mean hue, saturation, and value on top of RGB (separates color from brightness).

We test these individually and in combination (crop + normalize + HSV).

3.4 Models

Multiple Linear Regression (MLR) assigns one weight per feature plus an intercept. Prediction is a weighted sum. Fast, transparent, but assumes linearity.

K-Nearest Neighbors (KNN) finds the k closest houses in normalized feature space and predicts the average of their prices. Non-parametric, captures nonlinear patterns, but sensitive to k and the distance metric. We tune $k \in \{1, 3, 5, 7, 10, 15, 20, 30, 50\}$ and test four distance metrics (Euclidean, Manhattan, Chebyshev, Minkowski $p=3$). Based on our 500-trial shuffle cross-validation, we select Manhattan distance as our primary metric because it achieved the lowest median MAE across the sweep (\$129k at $k=1$, compared to \$131k for Euclidean at $k=1$). Manhattan distance sums absolute differences per feature, which is more robust to outliers in individual dimensions than Euclidean distance, which squares differences and gives disproportionate weight to extreme feature values.

3.5 Evaluation Metrics

All metrics are reported in dollars: MAE (primary), RMSE, SSE, median absolute error, max absolute error, and the fraction of predictions within \$50k, \$100k, and \$200k. We do not report R^2 because it is less interpretable for bounded, skewed price distributions.

4 Results

4.1 Specs-Only Baseline

Table 1 shows validation performance. Both models substantially outperform the constant predictor (training-set mean \$705,656), confirming that beds, baths, sqft, and city coordinates carry strong price signal.

Table 1: Validation metrics for specs-only models.

Model	MAE	RMSE	Within \$50k	Within \$100k	Within \$200k
Constant	\$280k	\$379k	13.4%	26.2%	46.1%
MLR (specs)	\$165k	\$237k	23.4%	45.6%	72.1%
KNN $k=15$ (specs)	\$150k	\$227k	29.5%	52.2%	76.3%

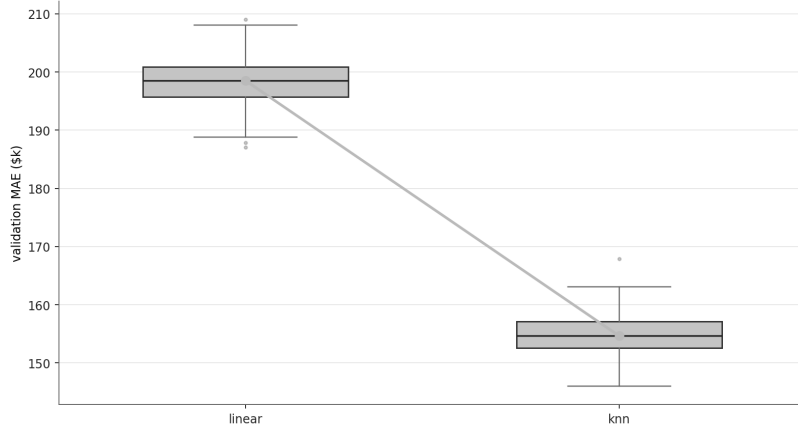


Figure 7: MAE distributions from 500 shuffle CV trials for specs-only models.

4.2 Adding Image Features

Table 2 adds the 27 simple image features. The result is unambiguous: image features do not improve either model. KNN degrades from \$150k to \$180k MAE; linear regression is essentially unchanged.

Table 2: Validation metrics with simple image features added.

Model	MAE	RMSE	Within \$50k	Within \$100k	Within \$200k
Constant	\$280k	\$379k	13.4%	26.2%	46.1%
MLR (specs)	\$165k	\$237k	23.4%	45.6%	72.1%
MLR (specs+img)	\$165k	\$236k	23.8%	45.4%	72.3%
KNN $k=15$ (specs)	\$150k	\$227k	29.5%	52.2%	76.3%
KNN $k=10$ (specs+img)	\$180k	\$258k	20.2%	40.3%	69.2%

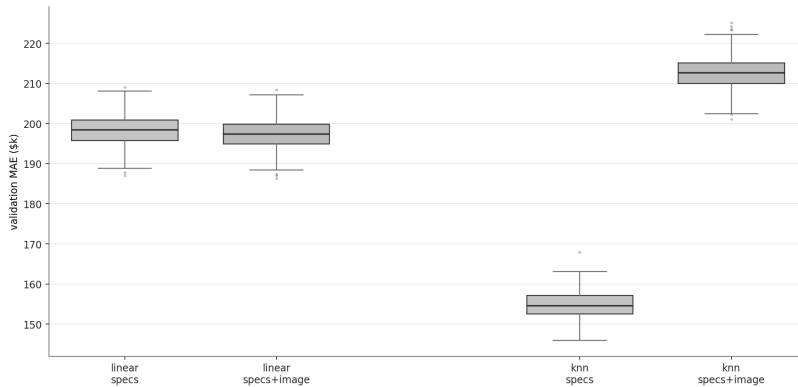


Figure 8: Shuffle CV MAE distributions: specs-only vs specs + image features.

4.3 Preprocessing Variants and Spatial Features

Table 3 shows validation MAE across preprocessing variants and the spatial filter features. No variant of image features improves upon the specs-only baseline. The 96 spatial filter features perform worse than specs alone for KNN and marginally differently for linear regression. Spatial-only features (without specs) perform near the constant predictor.

Table 3: Validation MAE across feature configurations. No image variant improves on specs-only.

Feature Set	Linear	KNN	Best
Specs Only	\$165k	\$150k	\$150k
Specs + Raw Photo	\$165k	\$181k	\$165k
Specs + Center Crop	\$165k	\$181k	\$165k
Specs + Brightness Norm	\$165k	\$181k	\$165k
Specs + Crop + Norm + HSV	\$165k	\$188k	\$165k
Specs + Spatial (96)	\$164k	\$218k	\$164k
Spatial Only	\$269k	\$265k	\$265k

4.4 Distance Metric Comparison and Stability

We tested four KNN distance metrics across 500 shuffle trials. Figure 9 shows the median MAE at each k value for all four metrics. Manhattan distance achieved the lowest median MAE across the sweep (\$129k at $k=1$), slightly outperforming Euclidean (\$131k at $k=1$). We select Manhattan distance for our final model based on this result. However, the overall pattern is the same across all four metrics: specs-only KNN converges to \$142 to \$156k at moderate k , and adding image features consistently increases error regardless of which distance function we use. The choice of metric does not change the conclusion about image features.

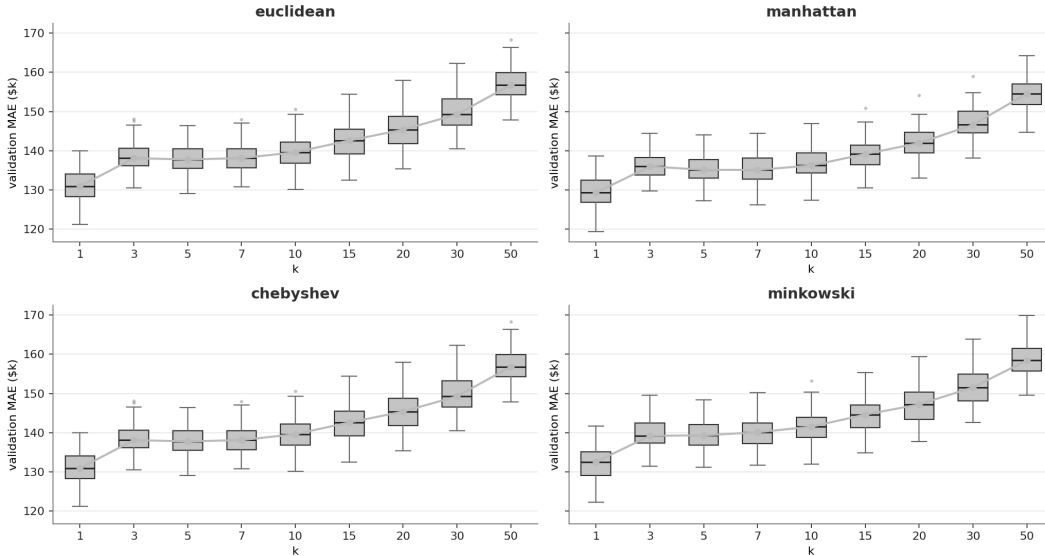


Figure 9: Median MAE across 500 shuffle trials for four distance metrics at varying k .

Table 4 confirms that the single-split results are not a fluke of random partitioning. The specs-only configurations consistently outperform specs+image across all 500 trials.

Table 4: Shuffle cross-validation summary (500 trials).

Configuration	Mean MAE	Median MAE	5th pctl	95th pctl
Linear, specs	\$198k	\$198k	\$192k	\$204k
Linear, specs+image	\$197k	\$197k	\$191k	\$203k
KNN, specs	\$155k	\$155k	\$149k	\$161k
KNN, specs+image	\$213k	\$213k	\$205k	\$219k

4.5 Per-City Subgroup Analysis

Table 5 compares specs-only vs. specs+image KNN for the top 10 cities. Most cities degrade when image features are added. Only Oceanside and Fontana show marginal improvement; La Quinta and Riverside worsen by over \$20k.

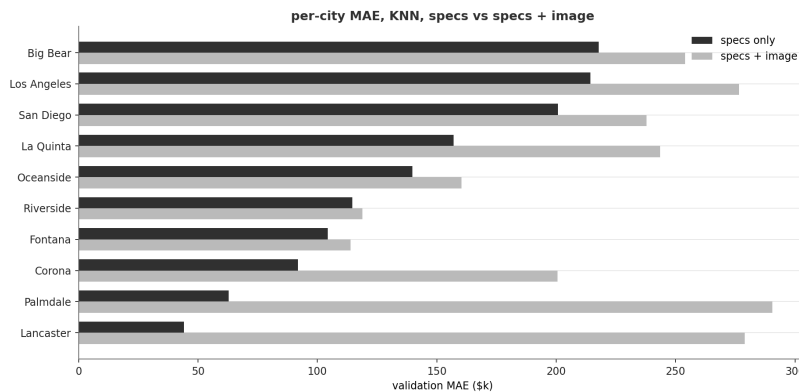


Figure 10: Per-city MAE delta (specs+image minus specs-only). Negative values indicate image features helped.

Table 5: Per-city validation MAE for the top 10 cities.

City	<i>n</i>	Specs	Specs+Img	Delta
Los Angeles	53	\$222k	\$232k	+\$10k
Big Bear	41	\$209k	\$211k	+\$3k
La Quinta	52	\$208k	\$238k	+\$31k
San Diego	106	\$189k	\$200k	+\$12k
Oceanside	35	\$112k	\$94k	-\$18k
Riverside	43	\$109k	\$132k	+\$24k
Fontana	40	\$84k	\$83k	-\$1k
Corona	33	\$72k	\$88k	+\$16k
Lancaster	46	\$38k	\$57k	+\$19k
Palmdale	33	\$36k	\$45k	+\$9k

4.6 Final Test Results

After locking all decisions on validation, we retrain on the combined training+validation set (12,379 rows) and evaluate once on the held-out test set (3,095 rows).

Table 6: Final test results. Best model: KNN $k=15$ Manhattan distance, specs-only.

Model	Features	MAE	RMSE	Within \$50k	Within \$100k
Constant	None	\$270k	\$366k	13.9%	27.1%
Linear	Specs	\$164k	\$233k	23.2%	44.5%
Linear	Specs+Spatial	\$163k	\$233k	23.1%	44.8%
KNN $k=15$	Specs	\$150k	\$224k	29.4%	51.0%
KNN $k=15$	Specs+Spatial	\$208k	\$291k	15.8%	33.9%

4.7 Clustering on Image Features

As a supplementary analysis, we apply K-means ($k=4$) to the 27 simple image features alone. If the photo carries price-relevant signal, clusters should separate by price band. The four clusters have mean prices of \$663k, \$664k, \$710k, and \$744k (Table 7), a spread of only \$81k, indicating that image features do not meaningfully separate price bands.

Table 7: K-means clusters ($k=4$) on simple image features.

Cluster	n	Mean Price
1	1,351	\$663k
2	1,812	\$664k
3	3,837	\$710k
4	3,058	\$744k

We also applied hierarchical clustering to the top 20 cities using their training-set mean prices and mean image features. The dendrogram shows that cities with similar mean prices cluster together regardless of their image statistics, further confirming that image features add little discriminative information.

5 Discussion

5.1 Why Image Features Did Not Help

The central finding is negative: hand-computed image features from small exterior photographs do not improve price prediction beyond beds, baths, sqft, and city coordinates. This holds across two model classes, four distance metrics, five preprocessing variants, 500 shuffle cross-validation trials, and a held-out test. Several factors contribute.

Photo quality and relevance. Some photos do not depict the house at all. They show landscaping, interior rooms, signage, or unrelated objects (Figure 11). When the photo is not of the house, every image-derived feature is noise.

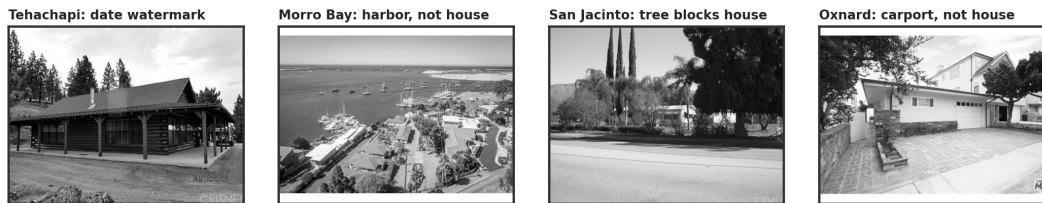


Figure 11: Examples of photos that do not clearly show the house exterior.

Feature scale mismatch. Image features (pixel means, edge densities, histogram bins) occupy fundamentally different dimensions than tabular features. For KNN, adding 27 or 96 features that carry no price signal dilutes the distance metric: two houses close in beds/baths/sqft/city may appear far apart because their photos differ in brightness or color histogram.

Lossy summarization. Collapsing a 128×128 image to 27 or 96 numbers discards most visual information. Mean brightness and edge density cannot distinguish a well-maintained house from a run-down one of the same color. The 4×4 pooled features retain some spatial layout but at too coarse a resolution for architectural detail.

The price cap. The \$2M ceiling clips the most expensive homes. Image features might help identify luxury properties, but the cap compresses the upper end, reducing the potential benefit.

Dimensionality. Each photo contains $128 \times 128 \times 3 = 49,152$ pixel values. Our 27 or 96 features compress this by factors of roughly 1,800 and 500, respectively. A 3×3 Sobel kernel captures local edge orientation but cannot represent high-level concepts like “brick facade” or “two-story layout.” The gap between raw pixels and our summary statistics is where the price-relevant visual information likely resides, and our hand-crafted pipeline cannot bridge that gap.

5.2 Why Specs Alone Are Sufficient

The specs-only KNN achieves \$150k MAE with 51% of predictions within \$100k, reasonable for a dataset spanning \$1.8M. Four tabular features capture the primary determinants: size and location. Within a city, square footage and room count explain most variation. The photo might encode condition, but this appears either already reflected in the specs or too subtle for hand-crafted features to detect.

5.3 Limitations

First, photos are small (128×128) and of variable quality; higher resolution might yield more signal. Second, feature extraction is entirely hand-crafted; CNNs could learn richer representations but fall outside course scope. Third, the dataset covers only Southern California in one time period; generalization is unknown. Fourth, we test only two model classes; ensembles might interact differently with image features, though the dilution problem would likely persist. Finally, the \$2M cap limits upside for any additional features on high-value homes.

6 Conclusion

We tested whether hand-computed image features from exterior photos can improve house price prediction beyond basic specifications. After evaluating two model classes, five feature configurations, four distance metrics, and 500 shuffle cross-validation trials, the answer is no. The best model, KNN with Manhattan distance at $k=15$ on specs alone, achieves a test MAE of \$150,186 with 51.0% of predictions within \$100,000. Adding image features increases KNN’s MAE by \$30 to 58k and leaves linear regression unchanged.

The hand-crafted features (color statistics, edge densities, spatially-pooled filter responses) do not capture the visual properties that distinguish expensive from inexpensive homes. The features are too lossy, the photos too small and variable, and the tabular features already too informative for the image signal to overcome the noise it introduces. Unless learned image representations (CNNs, pretrained embeddings) are available, the exterior photo adds computational cost without prediction benefit.

References

- [1] ted8080. (2023). House Prices and Images – SoCal. Kaggle. <https://www.kaggle.com/datasets/ted8080/house-prices-and-images-socal>
- [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
- [3] Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [4] Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- [5] Clark, A. (2015). Pillow (PIL Fork). <https://python-pillow.org/>